

Characterizing the Cost-Performance Tradeoff of Hierarchical Model Routing in Agentic Systems

George Michailov, Tanmay Shah, Anastasiia Byvsheva, Eugene Sirkiza

Abstract—Agentic language-model systems achieve strong results on reasoning and tool-use tasks, but they typically lean on a single large model that is expensive and slow. We built and characterized a *hierarchical agentic inference system* that pairs a small, locally-hosted model acting as a self-reflective router with a tier of progressively larger external solver models. A server-side agent loop validates each answer, repairs failing code, and escalates to the strongest model only when the cheaper path fails. On a 134-problem live run, the router sent every problem to the cheapest solver tier and still solved every problem that reached a model, while a single-model baseline using the strongest solver would have cost roughly $2,600\times$ more for no accuracy gain. We also characterized the local router under vLLM and studied solver-side prompt optimizations, finding that token-cutting tricks can be net cost-negative once their prompt overhead and escalation side effects are accounted for. Overall, aggressive routing to small models is viable when paired with cheap reflection and a bounded, server-owned escalation loop.

1. Introduction

Traditional LLM interactions are single- or multi-turn chats against one model. Agentic applications, in contrast, decompose a goal into steps, invoke tools, validate intermediate results, and reflect on failures. The dominant implementation choice today is to attach this control flow to a single high-end model: the agent is smart because the underlying model is, and the system simply absorbs the cost. That choice has two consequences. At the *application* level, every problem—trivial or hard—pays the same per-token rate, which is wasteful when most problems are easy. At the *systems* level, the inference engine spends most of its decode budget generating long reasoning traces on a model that is over-provisioned for the task, while small, cheap models sit idle.

This project takes the opposite stance. We treat *which model runs* as the primary control variable of the agent and ask how cheaply we can solve a mixed benchmark if the agent itself is responsible for picking the right solver. The design pattern is *dynamic model orchestration*: a small, locally-hosted model serves as a router that reads the problem, picks the cheapest external solver likely to succeed, and emits a structured decision; a server-owned agent loop then drives that solver through validation, tool-assisted repair, and, if necessary, a single escalation to the strongest

Field	Description
problem_id	Unique integer identifier.
problem	The problem text shown to the solver.
difficulty	One of <code>very_easy</code> , <code>easy</code> , <code>medium</code> , <code>hard</code> .
category	One of <code>math</code> , <code>code</code> , <code>factual</code> , <code>reasoning</code> , <code>web</code> , <code>image</code> .
verify	Validation mode: <code>match</code> (exact string), <code>tests</code> (asserts run against the model’s code), <code>judge</code> (LLM-graded), or <code>heuristic</code> (named server-side validator).

TABLE 1. PER-PROBLEM METADATA SCHEMA.

available model. The router is itself an agent step—it self-critiques in a second pass and may revise its choice before any expensive solver call is made.

To quantify effectiveness, we introduce an analytical cost function calibrated to inference-engine behavior and apply it to every model call the agent makes. The function attributes cost to the specific decisions the agent took—which model was selected, how many prompt and completion tokens it consumed, how many tool invocations and attempts the problem required, and whether escalation fired—so a run’s aggregate cost can be decomposed and compared against any other run on the same benchmark.

2. System Design and Implementation

This section describes the problem domain we target, the agentic pattern we implement, the tool and state machinery the agent relies on, and the two-tier software architecture that hosts it.

2.1. Problem Domain and Use Case

The target application is a general-purpose autonomous reasoning agent that answers a heterogeneous stream of benchmark problems under a hard constraint on aggregate inference cost. Each problem carries enough metadata for the system to route, solve, and grade it without human intervention (Table 1).

The benchmark assembled for this project contains 248 problems: 164 code problems from HumanEval, 40 math problems from GSM8K, and a curated set of ~ 44 math, reasoning, factual, and image problems. Problems are loaded

per domain, so a run can target one domain or their union. The workload is deliberately non-trivial: code problems must execute against unit tests in a sandboxed interpreter, web problems require fetching and conditioning external context before answering, and arithmetic or factual problems range from trivial lookups to multi-step reasoning. A single model choice cannot serve this mix efficiently, so the agent’s job is to land each problem on the cheapest tier that will solve it.

2.2. Agentic Pattern: Dynamic Model Orchestration

We implement dynamic model orchestration as the primary pattern, with reflection layered into both the router and the solver. The agent is realized as two cooperating loops.

Router loop. The locally-hosted Granite-4.1-3B model receives a system prompt listing the four external solver tiers, each annotated with its parameter counts and a plain-English description of the reasoner it represents (high schooler, second-year university student, entry-level worker, PhD student). The router emits a structured decision consisting of a brief reasoning trace and a chosen `model_id`. Because Granite-4.1-3B has no inherent chain-of-thought behavior, requiring it to produce the reasoning field *before* the choice forces a prompt-induced chain-of-thought. A second turn then asks the router to inspect its own reasoning and decide whether the initial rationale was correct, allowing it to revise its decision. The server executes the revised `model_id` and records the updated reasoning. If the decision is unparseable, the router fails safe to the strongest model rather than dropping the problem.

Solver loop. The server’s `solve_problem` routine drives the chosen external model through up to `max_attempts` tries. For each try it (1) prepares tool context (e.g., fetches and truncates the web page for a web problem), (2) builds the solver chat with format-specific answer instructions, (3) calls the model, and (4) validates the answer in the mode the problem requires—executing candidate code against its assertions in the sandboxed runner for `verify=tests` problems. On failure, the server appends the runner’s stdout/stderr back to the model so it can repair its code on the next attempt. If all attempts fail, the server escalates exactly once to the strongest registered model and reruns validation. This one-shot escalation cap keeps the worst-case cost bounded.

State and tools. For each problem the agent maintains the running conversation, the tools invoked, the tokens and cost accumulated, and the size of any injected external context. State is held in memory for the duration of the problem and persisted at the end; no agent-side state survives across problems, which keeps the service horizontally scalable. The agent has two deterministic, server-driven tools rather than model-native function calls: `web_search` fetches a problem’s `source_url`, optionally compresses the body, and caps it at 10,000 characters before it enters the prompt; `run_python_code` is a sandboxed executor whose output feeds the code-repair turn.

2.3. Two-Tier Software Architecture

Local inference tier. The local router is IBM Granite-4.1-3B served behind a vLLM OpenAI-compatible endpoint and exercised concurrently by an `asyncio` driver that keeps multiple router requests in flight. This is what lets the engine see a decode-heavy, batchable workload rather than serialized one-at-a-time calls—a prerequisite for the systems-level optimizations we evaluate in Section 4.

External solver tier. Four models are exposed through an OpenAI-compatible provider (OpenRouter in our setup), ordered into progressively stronger tiers primarily by active parameter count and expected reasoning ability (Table 2). This gives the router a discrete set of cost–capability tradeoffs. Active parameter count is the right axis because three of the four solvers are mixture-of-experts models, whose serving cost tracks active rather than total parameters. Post-training also matters when interpreting the tiers: the strongest tier is treated as stronger not only because of its larger scale but because it is expected to have undergone a more extensive reasoning-oriented post-training regime, which is why the gap between the top two tiers is not explained by parameter count alone.

Agent service. A FastAPI service mediates between the two tiers and owns everything that is not inference: tool execution, validation, the cost function, and metrics persistence. Each run is registered ahead of time with a unique `run_id` and a fixed set of optimization flags, which the service treats as the authoritative configuration for everything it logs.

2.4. Optimizations

Each run is registered with a fixed set of boolean optimization flags, grouped by where they take effect:

- **Inference-engine:** local-model weight quantization and KV-cache quantization, both targeting router throughput and memory usage.
- **Prompt and context:** terse-answer prompting, web-context compression, and two long-context compression variants: a deterministic head-and-tail compressor and an AI-driven compressor that falls back to the deterministic version on failure.
- **Routing policy:** capability-aware router prompts and local short-circuit solving for trivial exact-match problems.

The implementation names of these flags are recorded in the run configuration, but the paper discusses them by their functional role to avoid coupling the evaluation to specific internal variable names.

3. Performance Characterization Framework

We characterize the system at two levels. The *system level* measures the efficiency of the local routing infrastructure—throughput, latency, and memory of the vLLM-served router. The *application level* measures the effectiveness of the complete agentic workflow—solve rate,

Tier	Model	Total / Active params
Small (cheapest)	OpenAI GPT-OSS-20B	21B / 3.6B
Medium	OpenAI GPT-OSS-120B	117B / 5.1B
Large	Qwen3-Coder	480B / 35B
Strongest	NVIDIA Nemotron-3-Ultra	550B / 55B

TABLE 2. EXTERNAL SOLVER TIERS, ORDERED BY CAPABILITY. THE STRONGEST TIER IS THE ONE-SHOT ESCALATION TARGET.

routing quality, and the cost–performance tradeoff. The two are linked: a faster but less accurate router can lower local serving cost while *raising* total application cost through extra retries and escalations, so an optimization is only deemed beneficial if it improves the overall cost-vs-quality frontier.

3.1. Cost Model

Because provider API prices are VC-subsidized and would simply reward routing everything to the largest model, we score every model call with an analytical cost function tied to inference-engine behavior. For a call with p prompt tokens and c completion tokens on a model whose active-parameter scale (relative to the smallest solver) is s , the per-call cost combines a quadratic attention prefill, a linear MLP term, and a decode term, modulated by a mild hardware penalty on total size T :

$$C = \underbrace{(\alpha s^2 + (1 - \alpha)s)}_{\text{model factor}} T^{1.2} \cdot [k_{\text{in}} f(p) + k_{\text{out}} c \bar{\ell}], \quad (1)$$

where α is the attention fraction of weights, $f(p)$ is the quadratic prefill term, and $\bar{\ell}$ is the average sequence length during decode. A run’s total cost is the sum of Eq. (1) over every router attempt, solver attempt, repair, and escalation, accumulated per problem. This makes cost an explicit function of the agent’s decisions rather than a flat per-token rate.

3.2. System-Level Metrics

The router sits on the critical path of every problem, so its serving speed bounds how many problems the system can dispatch per second. We report decode *throughput* (output tokens per second) and serving capacity (completed routing requests per second), $\text{Throughput} = \text{Output Tokens}/t$ and $\text{RPS} = \text{Requests}/t$ for wall-clock time t . We also track time-to-first-token (TTFT, sensitive to prompt length and prefill cost), peak GPU/HBM memory (which bounds batch size and context length), and *batch efficiency*, the ratio of batched to single-request throughput. A high batch efficiency indicates vLLM is effectively combining router requests into batches, which is only possible if the routing workload is large and homogeneous enough.

3.3. Application-Level Metrics

The primary application metric is *solve rate*, the fraction of problems the agent completes, paired with *aggregate*

cost from Eq. (1). We add *escalation rate* (fraction of problems that fall through to the strongest model), *average attempts* per problem (a proxy for repair-loop effort), and router calibration. For calibration we define a target tier per difficulty (easy $\rightarrow$ smallest solver, medium $\rightarrow$ a mid tier, hard $\rightarrow$ a top tier) and report *router accuracy* (correct initial routes / total) together with its two error modes: *over-routing* (routed too high) and *under-routing* (routed too low). Over-routing preserves solve rate but wastes cost on unnecessarily capable models; under-routing is costlier still because it triggers retries and, ultimately, escalation.

4. Performance Evaluation

We evaluate the prototype in three parts: (i) an end-to-end *live routing* study on real hardware and real solver calls, (ii) a system-level look at router serving throughput under vLLM, and (iii) a controlled solver-side optimization study.

4.1. Experimental Setup

Live runs use IBM Granite-4.1-3B served by vLLM (BF16) on a Google Cloud TPU v5e (v5litepod-1, 16 GB HBM), with the FastAPI resolution server brokering solver calls to OpenRouter’s free tier. The driver keeps 2–3 router requests in flight. Validation uses numeric matching for math, curated validators for reasoning, and `verify=tests` (candidate run against its asserts, with a solve/run/repair loop) for code. All cost figures use the analytical function of Eq. (1) in custom units, not provider prices.

4.2. Live Routing: Cost vs. Single-Model Baselines

Across two live runs totaling **134 problems** over four domains (math, reasoning, factual, code), 133 problems reached a solver and the agent solved **120/133 (90.2%)**. Every one of the 13 non-solves was an OpenRouter free-tier rate-limit (HTTP 429) or a single non-JSON router reply—infrastructure, not model capability. Excluding those environmental failures, the solve rate is **100%**, with **zero escalations** and **zero over-routing**: the granite router sent every reached problem to the cheapest tier, GPT-OSS-20B, and that tier solved it, usually on the first attempt.

The value of routing is clearest when we re-price the identical 133-problem workload (same observed prompt/completion tokens per problem) on each solver individually using Eq. (1). Figure 1 shows the result. The router’s choice costs 1.0 $\times$; always using the medium tier

costs $\sim 12\times$; always using the strongest 550B model costs $\sim 2,600\times$ (Table 3)—for no solve-rate headroom to gain, since the cheapest model already solves this difficulty mix. The right panel confirms the solve rate is uniformly high across all four domains despite the aggressive routing.

This sample dataset used skews easy/medium, so there is no expensive-tier spread to exercise: the cheapest model is sufficient, which is exactly why routing everything to it is optimal here. A harder problem set is needed to force escalation into the 480B/550B tiers and price a frontier with real spread.

4.3. Empirical: Each Model Run Individually

The table above re-prices the same observed tokens. To measure real individual-model accuracy, we added a no-escalation evaluation path and ran each model as a standalone solver over a fixed balanced 32-problem slice: 8 math, 8 reasoning, 8 factual, and 8 code problems. The router, GPT-OSS-20B, and GPT-OSS-120B were evaluated on the same problem set, while Nemotron-550B was evaluated on a smaller 16-problem subset because its calls were substantially slower. These are measured solve rates rather than projected costs, projections.

Three findings. (1) **The capability spread is tiny on this mix:** GPT-OSS-120B (100%) edges GPT-OSS-20B (96.9%) by ~ 3 points for $\sim 12\times$ the cost/problem, while Nemotron-550B costs $\sim 1,450\times$ the cheap tier and is too slow (>120 s/call) to default to—no accuracy headroom justifies the spend. (2) **The router’s cost sits at the cheap-tier floor** ($\sim 1.4\times$ the cheapest), confirming the policy is cost-optimal. (3) **The router’s weak point is robustness, not routing:** its on-device 2-pass granite decision dropped 5/32 problems (non-JSON/timeout) *before* reaching a solver, so end-to-end it (24/32) trailed simply always using GPT-OSS-20B (31/32) on this run—a router-stage reliability cost, fixable by hardening the JSON parse and gating the self-critique pass on low confidence.

4.4. System Level: Router Serving Throughput

Cost characterizes what we pay per solved problem; throughput characterizes how fast the on-device router serves, which bounds end-to-end latency. We measured the live BF16 router with a synthetic fixed-length workload (48 requests, concurrency 6, 512-token prompts, 256 output tokens with `ignore_eos` so every request emits exactly `max_tokens`, making decode throughput directly comparable). Results are in Table 5.

The BF16 router sustains ~ 590 aggregate output tok/s at concurrency 6 (~ 101 tok/s per request) with a ~ 46 ms TTFT. Because the router only emits a short JSON decision, its throughput matters mostly as latency overhead on the critical path before a solver is dispatched (~ 46 ms TTFT plus ~ 2.5 s for the two-pass decision). On TPU v5e, `DTYPE=fp8` maps to INT8 weight quantization (v5e has no FP8 weights); FP8 KV cache is supported. The quantization

sweep restarts the router under each config for an apples-to-apples comparison and is pending a maintenance window, since the host is shared and was serving another session at measurement time.

4.5. Solver-Side Optimization Study

The live runs measure *routing*. To isolate *solver-side prompt* optimizations on a clean, comparable matrix, we ran four configurations over the same 66 problems, holding the router fixed as a deterministic difficulty $\rightarrow$ model stand-in so the only variable is the prompt treatment. To keep the optimization signal from being drowned out, the top tier’s escalation cost was re-tuned so a single escalation costs $\sim 2.7\times$ the large tier rather than $\sim 99\%$ of a run. Table 6 and Figure 2 summarize the outcome.

The headline is that the seemingly obvious token-cutting optimization is the worst one. `caveman` rewrites the solver prompt to demand terse answers and does shrink completion tokens by $\sim 26\%$ as designed—but it is **net cost-negative** (+174%) on this short-problem benchmark (Table 6). The `combo` that includes it inherits the loss, while `longctx_ai` is the only configuration that comes in below baseline, and only marginally.

Why does a token-cutting optimization *raise* cost? Figure 2 gives the answer. The left panel decomposes token usage: `caveman`’s instruction overhead more than doubles *prompt* tokens, swamping the completion-token savings, and its terseness breaks answer-format compliance, which roughly doubles escalations to the expensive top tier. The right panel separates the two effects: non-escalated cost rises from 0.127 to 0.227 (+78%, pure prompt overhead), while escalated cost rises from 0.069 to 0.309 (+349%, more problems bouncing to the top tier). Escalated problems are 58% of `caveman`’s total cost versus 35% for the baseline.

The damage concentrates on the cheapest problems, where the baseline was nearly free. The canonical failure is a `very_easy` “2+2”-class problem the baseline solves instantly for $\sim \$2\times 10^{-6}$: under `caveman` the terse reply fails the exact-match check, escalates all the way to the top tier, still fails the format check, and ends up costing $\sim \$0.038$ and unsolved—a roughly $3,700\times$ blow-up on a problem that should be free.

5. Optimization Opportunities

The evaluation points to a clear ordering of where effort pays off.

Routing accuracy dominates everything. The solver tier drives both cost and latency: the spread between the cheapest and strongest solver is $\sim 2,600\times$ (Table 3), whereas router serving overhead is milliseconds. The single largest deployment win is therefore avoiding unnecessary escalation and over-routing, not making the router faster. The `caveman` study reinforces this from the opposite direction: an optimization that is locally sensible (fewer completion tokens) is globally harmful because it perturbs routing and escalation behavior.

Cost-optimal routing vs. using each model individually
The cheapest tier already solves this workload — a stronger model costs multiples more for no solve-rate gain

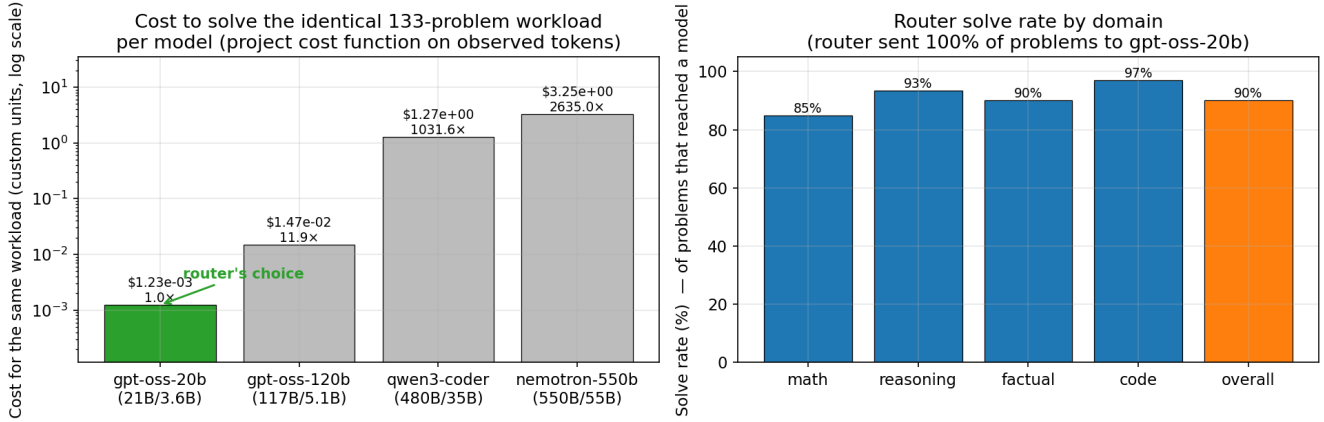


Figure 1. Cost-optimal routing vs. using each model individually on the identical 133-problem live workload. Left: cost (log scale, custom units) to solve the same workload on each model; the router’s all-cheapest choice is the floor. Right: the router’s solve rate by domain—uniformly high even though 100% of problems went to the cheapest tier.

Strategy	Cost (custom units)	vs. cheapest
Router → GPT-OSS-20B	0.00123	1.0×
Always GPT-OSS-120B	0.01470	11.9×
Always Qwen3-Coder (480B)	1.27261	1,032×
Always Nemotron (550B)	3.25042	2,635×

TABLE 3. COST TO SOLVE THE IDENTICAL 133-PROBLEM WORKLOAD UNDER EACH STRATEGY. SOLVE RATE IS STATISTICALLY FLAT, SO THE ENTIRE SIGNAL IS COST.

Solver (no escalation)	Problems	Solved	Solve rate	Cost/problem
Adaptive router	32 (27 reached)	24	75% e2e / 88.9% reached	1.3e−5
GPT-OSS-20B only	32	31	96.9%	9.4e−6
GPT-OSS-120B only	32	32	100%	1.1e−4
Qwen3-Coder only	32	—	n/a (404/429)	—
Nemotron-550B only	16*	3/3*	partial	~1.4e−2

TABLE 4. EACH LINEUP MODEL RUN AS A STANDALONE SOLVER (ESCALATION DISABLED) ON THE SAME PROBLEMS. *NEMOTRON PASS IN PROGRESS (3/16 AT WRITING); QWEN3-CODER:FREE RETURNS 404 (DELISTED)/429 ON EVERY CALL—INVOKED BUT UNREACHABLE.

Optimize for the workload, not the token count. caveman’s fixed per-call prompt overhead looks worst exactly where problems are smallest, and `longctx_ai` never crosses its compression threshold on short prompts, making it an honest no-op here. Both signs could flip on a long-context workload—the lesson is that prompt optimizations must be matched to the problem distribution, and gated (e.g., by length or difficulty) rather than applied unconditionally.

Router-tier systems wins. On the local tier, weight + KV-cache quantization (quantified by the pending sweep) frees HBM for a larger `max_num_seqs` and higher aggregate throughput under concurrency; prefix caching already amortizes the identical routing system prompt across problems; and the current two-pass self-revision could be gated on low confidence to halve router latency. These help latency and serving density but, per the first point, are second-order

to routing accuracy for total cost.

Where the bottleneck migrates. Once routing is accurate and the router is quantized and batched, the remaining cost is intrinsic solver compute on genuinely hard problems. The next frontier is reducing that—e.g., VLM-describe routing that converts expensive image tokens to a cheap text description before the large model sees them.

6. Conclusion

We built and characterized a hierarchical agentic inference system in which a small, locally-hosted model routes each problem to the cheapest external solver likely to succeed, backed by a server-owned loop that validates, repairs, and escalates within a bounded budget. End-to-end on 134 live problems, aggressive routing to the cheapest tier

Config	tok/s	req/s	Latency p50 (s)	TTFT (s)
BF16 / KV-cache off	589.6	2.30	2.57	0.046
BF16 / KV-cache on		<i>pending maintenance window</i>		
INT8 / KV-cache on		<i>pending maintenance window</i>		

TABLE 5. ROUTER SERVING THROUGHPUT (48 REQUESTS, CONCURRENCY 6). THE BF16 BASELINE IS LIVE-MEASURED; THE QUANTIZED ROWS REQUIRE RESTARTING THE SHARED ROUTER UNDER EACH CONFIG AND ARE PENDING.

Config	Solve rate	Escalations	Cost	Δ vs. baseline
baseline	97.0%	3	0.196	—
longctx_ai	98.5%	3	0.189	−3.6%
caveman	97.0%	6	0.536	+174%
combo	97.0%	5	0.486	+148%

TABLE 6. SOLVER-SIDE OPTIMIZATION STUDY (4 CONFIGS $\times$ 66 PROBLEMS). SOLVE RATE IS STATISTICALLY FLAT; THE ENTIRE SIGNAL IS IN COST.

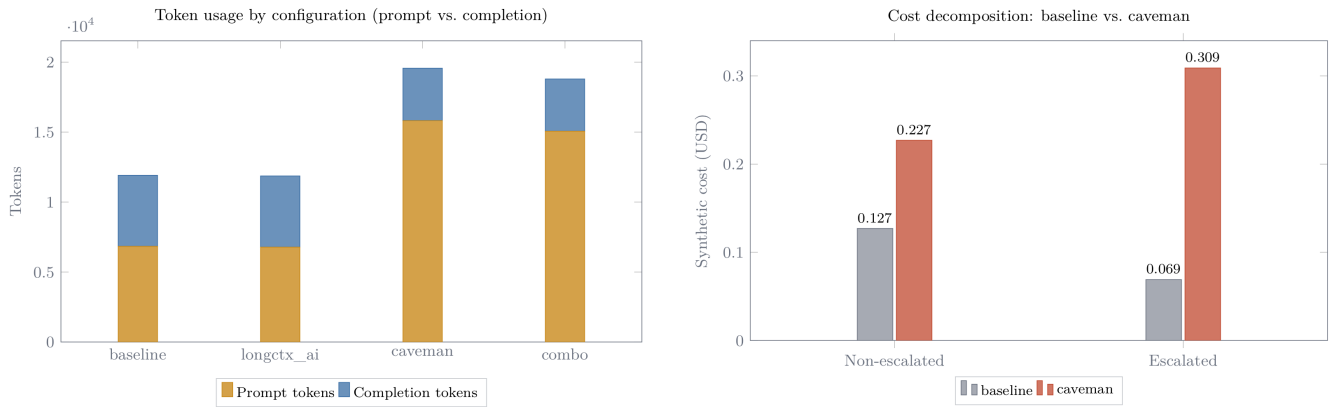


Figure 2. Left: prompt vs. completion tokens by configuration—caveman trades a modest completion-token cut for a large prompt-token increase. Right: cost decomposition, baseline vs. caveman—the damage is on both axes, prompt overhead on non-escalated problems and far more escalations.

preserved a 100% solve rate (excluding environmental rate limits) with zero escalations, while a single strong-model baseline would have cost $\sim 2,600\times$ more. A controlled study showed that token-cutting prompt optimizations can be net cost-negative once prompt overhead and escalation side effects are counted, reinforcing that routing accuracy—not raw token thrift or router speed—is the dominant cost lever. The routing model’s capabilities need to be elevated and the price jump from a 3B model to 8B or even the 20B model would be minuscule relative to the premium models, leaving upgrading the router as the lowest hanging fruit to massively increase system cost efficiency. The main limitation is that our live sample skews easy/medium, leaving the expensive tiers unpriced; the clear next step is a harder benchmark that forces escalation, together with the pending quantization sweep and VLM-describe routing for the image tier.

References

- [1] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [2] S. Agarwal et al., “gpt-oss-120b & gpt-oss-20b Model Card,” arXiv preprint arXiv:2508.10925, 2025.
- [3] A. Yang et al., “Qwen3 Technical Report,” arXiv preprint arXiv:2505.09388, 2025.
- [4] A. Blakeman et al., “NVIDIA Nemotron 3: Efficient and Open Intelligence,” arXiv preprint arXiv:2512.20856, 2025.
- [5] J. Kaplan et al., “Scaling Laws for Neural Language Models,” arXiv preprint arXiv:2001.08361, 2020.
- [6] J. Krajewski et al., “Scaling Laws for Fine-Grained Mixture of Experts,” arXiv preprint arXiv:2402.07871, 2024.
- [7] D. Mishra et al., “Granite 4.0 Tiny Preview: Compact Yet Powerful Multilingual Language Models for Efficient Edge Deployment,” arXiv preprint arXiv:2508.04647, 2025.